GÜNLÜK YAZILIM GELİŞTİRME AKIŞLARINDA BÜYÜK DİL MODELLERİ KULLANARAK HATA ŞİDDETİ TAHMİNİ

Reşit DEMİR

Havvanur DERVİŞOĞLU
Ruşen HALEPMOLLASI
Hakan GÜNDÜZ

Sunum Akışı

- Problem ve motivasyon
- Amaç, kapsam ve katkılar
- Veri kümesi ve önişleme
- Özellik çıkarımı (metin + kod + metrik)
- Öğrenme paradigmaları (zero-shot, few-shot, fine-tuning, klasik)
- Deney düzeni ve değerlendirme ölçütleri
- Sonuçlar, çıkarımlar, gelecek çalışmalar

Problem ve Motivasyon

- Artan proje karmaşıklığı → hata tespiti ve önceliklendirme zorlaşıyor
- Şiddet (severity) yanlış belirlenirse: kaynak israfı, gecikme, kritik hataların kaçması
- Hata kaydı metni + commit/kod bağlamı birlikte analiz edilirse daha isabetli karar mümkün
- LLM/Transformer tabanlı temsiller, yalnızca anahtar kelimeye değil bağlama dayanır

Amaçlar

- Hata şiddetini erken aşamada tahmin ederek önceliklendirmeyi kolaylaştırmak
- Riskli alanları hızlı tespit edip yayın/teslimat süresini kısaltmak
- Manuel sınıflandırma eforunu ve insan kaynaklı hataları azaltmak
- CI/CD süreçlerine entegre edilebilir pratik bir çerçeve sunmak

Kapsam

- Odak: hata raporu/commit bağlamından şiddet tahmini
- Metin alanları: hata raporu özeti, commit mesajı, kod açıklamaları
- Sayısal alanlar: statik kod metrikleri (karmaşıklık, bağımlılık, bakım vb.)
- Çıktı: 3 sınıf (Low/Medium/High) — kritik sınıf High ile birleştirildi
- Deneyler: Python/Jupyter; sonuçlar bağımsız test kümesinde raporlandı

Bu Tezin Katkıları

- Tek-modlu yaklaşımlara karşı metin+kod+metrikleri birleştiren hibrit bir boru hattı
- Zero-shot baseline ile denetimli modellerin karşılaştırmalı analizi
- Topluluk (ensemble) + eşik ayarı ile dengesiz sınıflarda Macro-F1 iyileştirmesi
- Operasyonel bakış: CI/CD entegrasyonu için uygulanabilir bir akış önerisi

Hata Şiddeti Sınıfları

- Etiket uzayı: Low / Medium / High
- Kritik hatalar daha sağlam tahmin için High ile birleştirilmiştir
- Amaç: dengesizlik etkisini azaltmak ve karar sınırlarını netleştirmek
- Çıktı: sınıf + olasılık (confidence) üretimi

Literatür: Geleneksel SDP ve Sınırlar

- İlk SDP çalışmaları çoğunlukla elde hazırlanmış yazılım metriklerine dayanır (ör. McCabe, CK).
- Metrikler yorumlanabilir ve hesaplaması kolaydır; klasik ML modelleriyle yaygın kullanılmıştır.
- Ancak tek başına metrikler, kodun anlamsal bağlamını ve hata raporu dilini sınırlı yakalar.
- Literatürde birçok çalışma kusur var/yok tahmini veya modül hatalılığına odaklanır; şiddet daha az ele alınır.
- Etiketleme farklılıkları ve sınıf dengesizliği, şiddet tahmini çalışmalarında temel zorluklardandır.
- Bu sınırlılıklar, bağlamı zengin temsiller ve çoklu veri kaynaklarını birleştiren yaklaşımları motive eder.

Literatür: Derin Öğrenme, Transformer ve Boşluk

- Derin öğrenme ile SDP'de temsil öğrenimi öne çıkar; metin/koddan otomatik özellik çıkarımı mümkün olur.
- Transformer tabanlı modeller (ör. CodeBERT), kod ve doğal dili birlikte ön-eğiterek zengin gömüler üretir.
- Hata raporu açıklaması, commit mesajı ve kod parçaları şiddet tahmini için güçlü bağlamsal sinyaller sağlayabilir.
- İki yaygın strateji: dondurulmuş kodlayıcıyla özellik çıkarımı ve seçili katmanlarla ince ayar (fine-tuning).
- Sadece metin ya da sadece metrik kullanımı eksik kalabilir; hibrit/çok-modlu temsiller daha dengeli sonuç verir.
- Bu tez, zero-shot, denetimli, ensemble ve fine-tuning paradigmalarını karşılaştırarak bu boşluğu sistematik inceler.

YÖNTEM VE MIMARI

Tezde, metin + kod + metrikleri birleştiren çok-modlu bir şiddet tahmin hattı kurulmuştur.

Genel Yaklaşım

- Girdi: hata raporu metni, commit mesajı, diff/kod, statik metrikler
- Önişleme: temizleme, tokenleştirme, metrik ölçekleme (fit: train, transform: test)
- Özellik çıkarımı: TF-IDF + CodeBERT gömüleri + metrik vektörü
- Modelleme: zero-shot baseline + denetimli klasik modeller + fine-tuning
- Çıktı: şiddet sınıfı ve güven/olasılık

Veri Kümesi

- Commit düzeyinde oluşturulmuş kayıtlar: her kayıt hem metinsel hem sayısal alan içerir
- Metin alanları: code, code_comment, code_no_comment (gerektiğinde commit mesajı vb.)
- Sayısal alanlar: satır sayısı, karmaşıklık, bağımlılık ve bakım metrikleri
- Eğitim: train_scaled.csv | Bağımsız test: test_scaled.csv

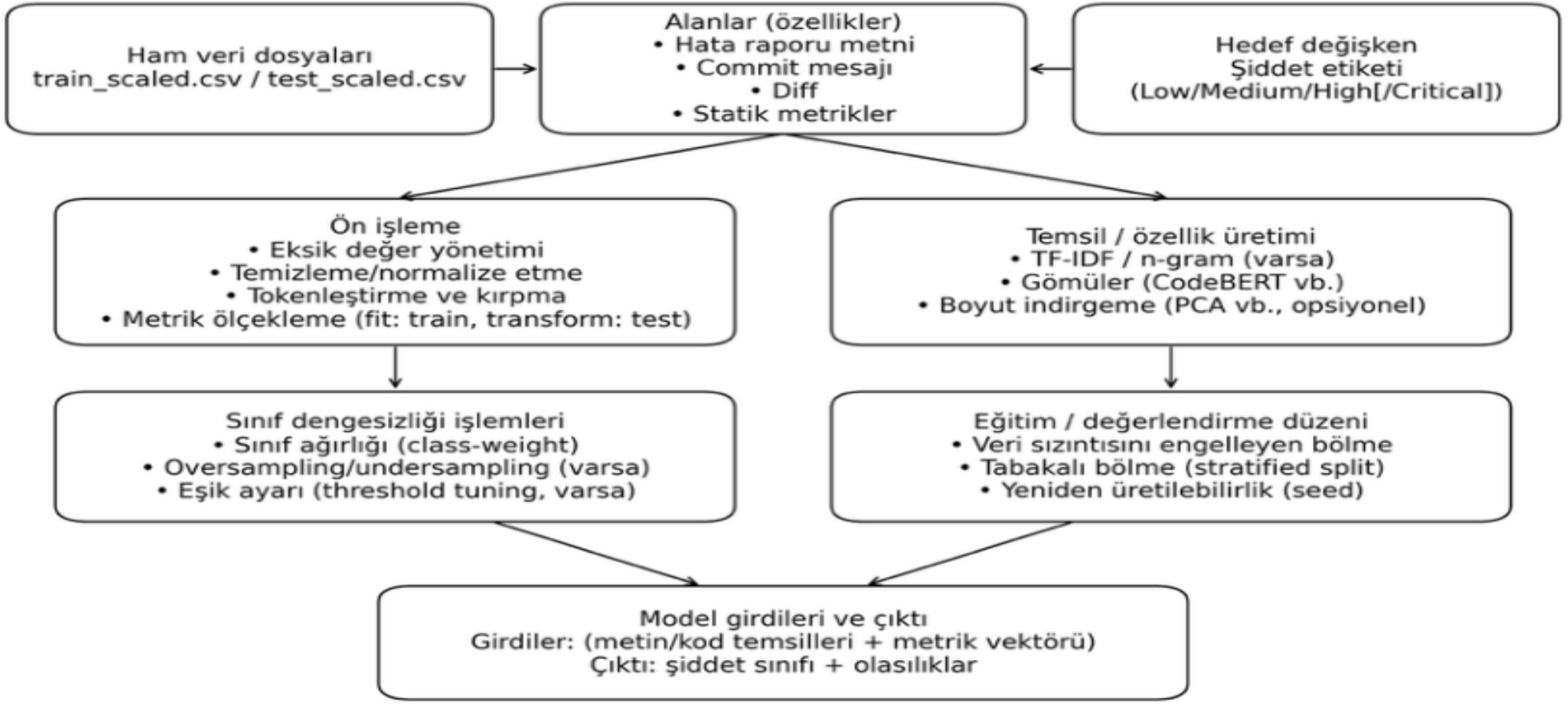
Sınıf Dengesizliği

- Test kümesi sınıf dağılımı dengesiz: Low=47, Medium=294, High=161
- Bu nedenle doğruluk tek başına yeterli değil; Macro-F1 temel karşılaştırma ölçütü
- Dengesizlik için: class-weight, oversampling/undersampling, eşik ayarı seçenekleri

Ön İşleme Adımları

- Eksik/tutarsız kayıtların elenmesi; hedef etiketi olmayan örneklerin çıkarılması
- Metinsel alanlarda normalize etme ve kırpma (uzunluk sınırı)
- Metriklerde z-skor standardizasyonu (büyük ölçekli değişken etkisini dengeleme)
- Deney tekrarlanabilirliği için sabit seed ve stratified split

Veri Ön İşleme ve Akış Şeması



Metinsel Özellikler (TF-IDF)

- Hata raporu ve commit mesajından TF-IDF vektörleri
- N-gram (varsa) ile kelime dizisi bağlamını güçlendirme
- Avantaj: hızlı, yorumlanabilir; dezavantaj: bağlamı sınırlı
- Denetimli modeller için güçlü bir taban özellik uzayı

Kod/Metin Temsilleri (CodeBERT)

- CodeBERT: kod + doğal dil üzerinde önceden eğitilmiş Transformer
- Bu tezde: (1) sabit gömü çıkarımı, (2) fine-tuning senaryosu
- Amaç: diff/kod semantiğini ve açıklama bağlamını yakalamak
- Hibrit özellik uzayında TF-IDF ve metriklerle birlikte kullanılır

Sayısal Kod Metrikleri

- Örnek metrikler: lc, pi, ma, nbd, ml, d, mi, fo, r, e
- Farklı ölçeklerdeki metrikler z-skor ile standardize edilir
- Amaç: satır sayısı gibi büyük değerlerin modeli orantısız etkilemesini önlemek
- Metrikler tek başına ve gömülerle birleştirilmiş çok-modlu temsilde kullanılır

Çok-Modlu Birleştirme

- Metin/kod temsilleri + metrik vektörü tek bir hibrit özellik uzayında birleştirilir
- Basit yaklaşım: vektör birleştirme (concatenation)
- Opsiyonel: PCA vb. ile boyut indirgeme (maliyet/overfit azaltma)
- Beklenti: High gibi kritik sınıfları ayırmada ek sinyal

ÖĞRENME PARADIGMALARI

Etiketli veri miktarına göre üç ana yaklaşım değerlendirildi: zero-shot, few-shot, fine-tuning ve klasik taban modeller.

Zero-shot (Prompt/NLI) Baseline

- Etiketli veri olmadan, doğal dil çıkarımı (NLI) ile sınıf olasılıklarını kıyaslama
- Her sınıf için metinsel tanımlar (label descriptions) kullanılır
- Avantaj: hızlı başlangıç ve düşük kurulum maliyeti
- Dezavantaj: dengesiz sınıflarda (Low/High) Macro-F1 düşebiliyor → baseline

Zero-shot: Etiket Tanımları Örneği

```
self.severity_labels = {
            0: "Low severity bug: Minor issue with minimal impact on functionality. Simple fix required. Low priority.",
            1: "Medium severity bug: Moderate issue affecting some functionality. Needs attention but not critical. Medium priority.",
            2: "High severity bug: Major issue significantly impacting important features. Requires urgent fix. High priority.",
            3: "Critical severity bug: Severe issue causing system failures or security risks. Immediate fix mandatory. Critical priority."
        }
```

Şekil 4.4.1. Örnek etiketlerin kullanımı ekran çıktısı

Denetimli Modeller (Klasik ML)

- Hibrit özellik uzayı üzerinde: LogReg_L2, LogReg_L1, Calibrated_SVC
- Amaç: etiketli veri ile karar sınırını proje verisine uyarlamak
- Class-weight ve olasılık kalibrasyonu ile dengesizliği azaltma
- En iyi sonuçlar topluluk modelinde gözlendi

Topluluk (Ensemble) ve Eşik Ayarı

- Ensemble: birden fazla taban sınıflandırıcının çıktılarını birleştirme
- Eşik ayarı: sınıf olasılıklarına çarpan uygulayarak dengesizliğe duyarlılık
- Örnek eşik çarpanları: [1.2, 1.0, 1.3] (Low, Medium, High)
- Hedef: Low/High sınıflarını daha iyi yakalarken Macro-F1'i artırmak

Fine-tuning (Tam İnce Ayar) Senaryosu

- Modelin tüm parametreleri (ya da büyük kısmı) etiketli veriyle güncellenir
- Optimizasyon: AdamW, erken durdurma, öğrenme oranı azaltma
- Opsiyonel: LoRA benzeri düşük dereceli uyarlama ile bellek maliyeti azaltma
- Bu tezde CodeBERT fine-tuning ile 4 sınıflı senaryo da raporlandı

DENEYLER VE DEĞERLENDIRME

Tüm modeller aynı eğitim/test bölmesi üzerinde değerlendirilerek karşılaştırılabilir sonuçlar elde edilmiştir.

Deney Ortamı

- Kütüphaneler: PyTorch, Transformers, scikit-learn, pandas, NumPy
- Tekrarlanabilirlik: sabit seed ve aynı veri kümeleri
- Amaç: zero-shot, few-shot ve fine-tuning sonuçlarını karşılaştırılabilir kılmak

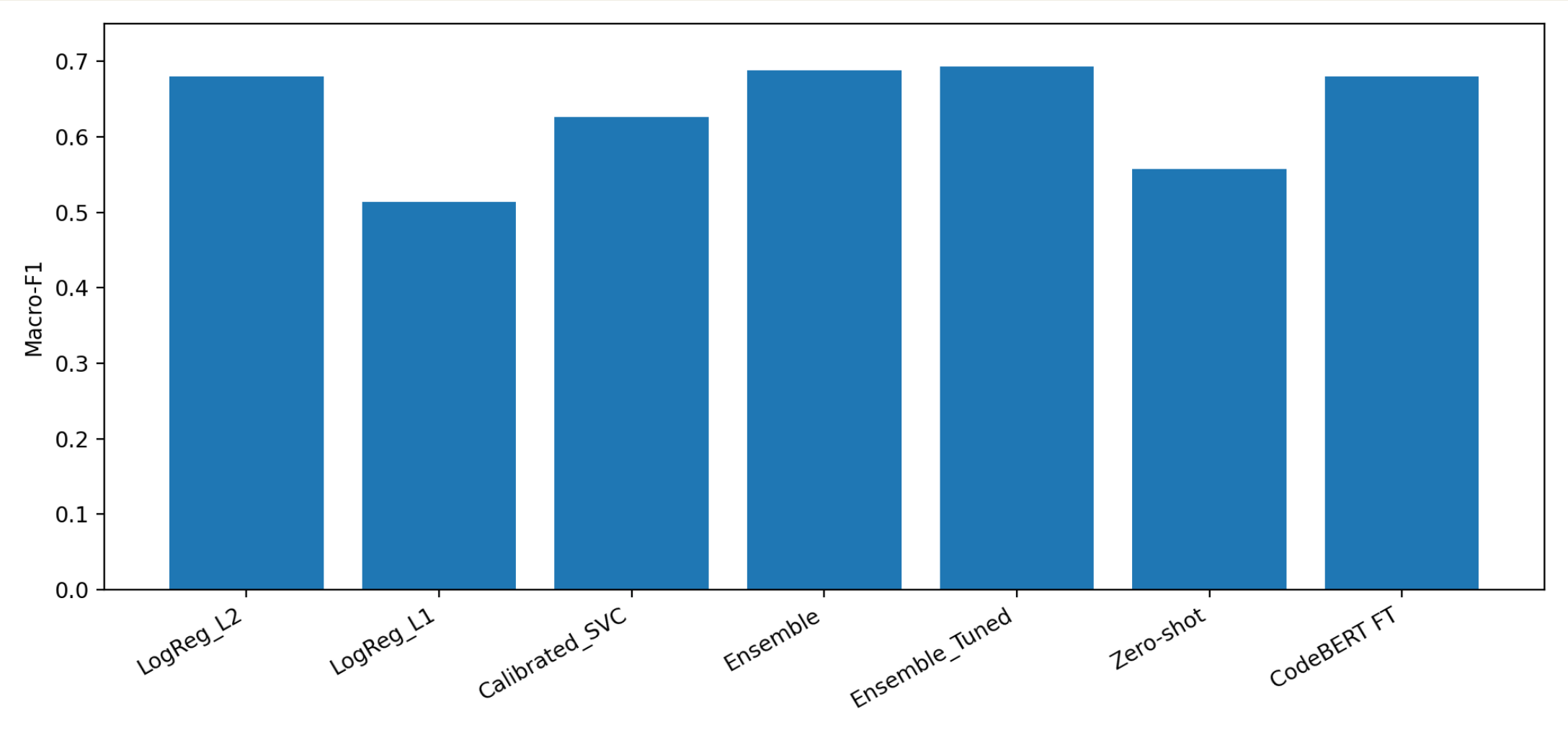
Değerlendirme Ölçütleri

- Accuracy = (TP+TN)/N
- Precision = TP/(TP+FP) | Recall = TP/(TP+FN)
- F1 = 2·P·R/(P+R)
- Macro-F1: sınıf F1 ortalaması (dengesiz veri için ana ölçüt)
- Weighted-F1, MCC ve Confusion Matrix ek göstergeler

Sonuçlar: Model Karşılaştırması (Tablo 7.2.1)

Paradigma	Model	Sınıf	Accuracy	Macro-F1
Few-shot	LogReg_L2	3	0.7151	0.6804
Few-shot	LogReg_L1	3	0.5418	0.5142
Few-shot	Calibrated_SVC	3	0.7371	0.6265
Few-shot	Ensemble	3	0.7390	0.6883
Few-shot (Threshold)	Ensemble_Tuned	3	0.7311	0.6933
Zero-shot (Hybrid)	Improved Severity Predictor	3	0.6753	0.5572
Fine-tuning	LLM CodeBERT	4	0.7590	0.6800

Makro-F1 Karşılaştırması (Grafik)



Bulguların Yorumu

- Zero-shot: hızlı başlangıç sağlar ancak dengesizlikten etkilenir (özellikle Low/High)
- Hibrit özellik + denetimli öğrenme → Macro-F1 artışı
- Ensemble yaklaşımı: tek modele göre daha kararlı ve dengeli sonuç
- Eşik ayarı: nadir sınıflarda duyarlılığı artırabilir

Sınırlılıklar

- Veri kümesi belirli projelerden türetildi → genelleme sınırı olabilir
- Şiddet etiketleri manuel/etiketleme kalitesi etkisi
- Metriklerin alan bağımlılığı: bazı metrikler her projede aynı sinyali taşımayabilir
- Fine-tuning maliyeti: donanım ve hiperparametre duyarlılığı

Gelecek Çalışmalar

- Daha fazla projeden veri ile çeşitliliği artırma
- Aktif öğrenme ve örnek seçimi ile etiketleme maliyetini düşürme
- Kuruma/projeye özel küçük fine-tuning (LoRA) ile uyarlama
- Hata takip araçları ve CI/CD pipeline’a canlı entegrasyon (feedback loop)

Teşekkürler

- Katılımınız ve değerli sorularınız için teşekkür ederim.
- Sorularınızı alabilirim.